Aim :- To design stimulate and verify the magnification functionality of various fundamental combinational logic circuits using verilog HDL.

Theory :-

This experiment covers the Design And verification of standard combinational circuits and fundamental concepts of the verilog language.

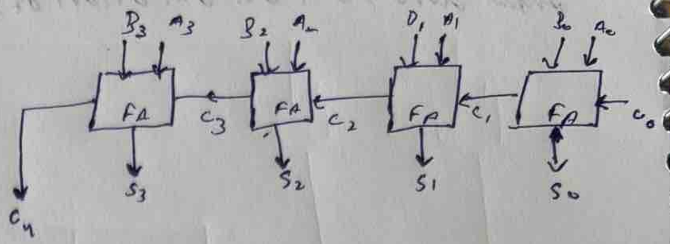
Tools required :-

Model Sim :- This is an HDL simulation tool used for verifying the functionality of digital circuits.

Verilog HDL :- This is the language used to describe the digital circuits and their test benches.

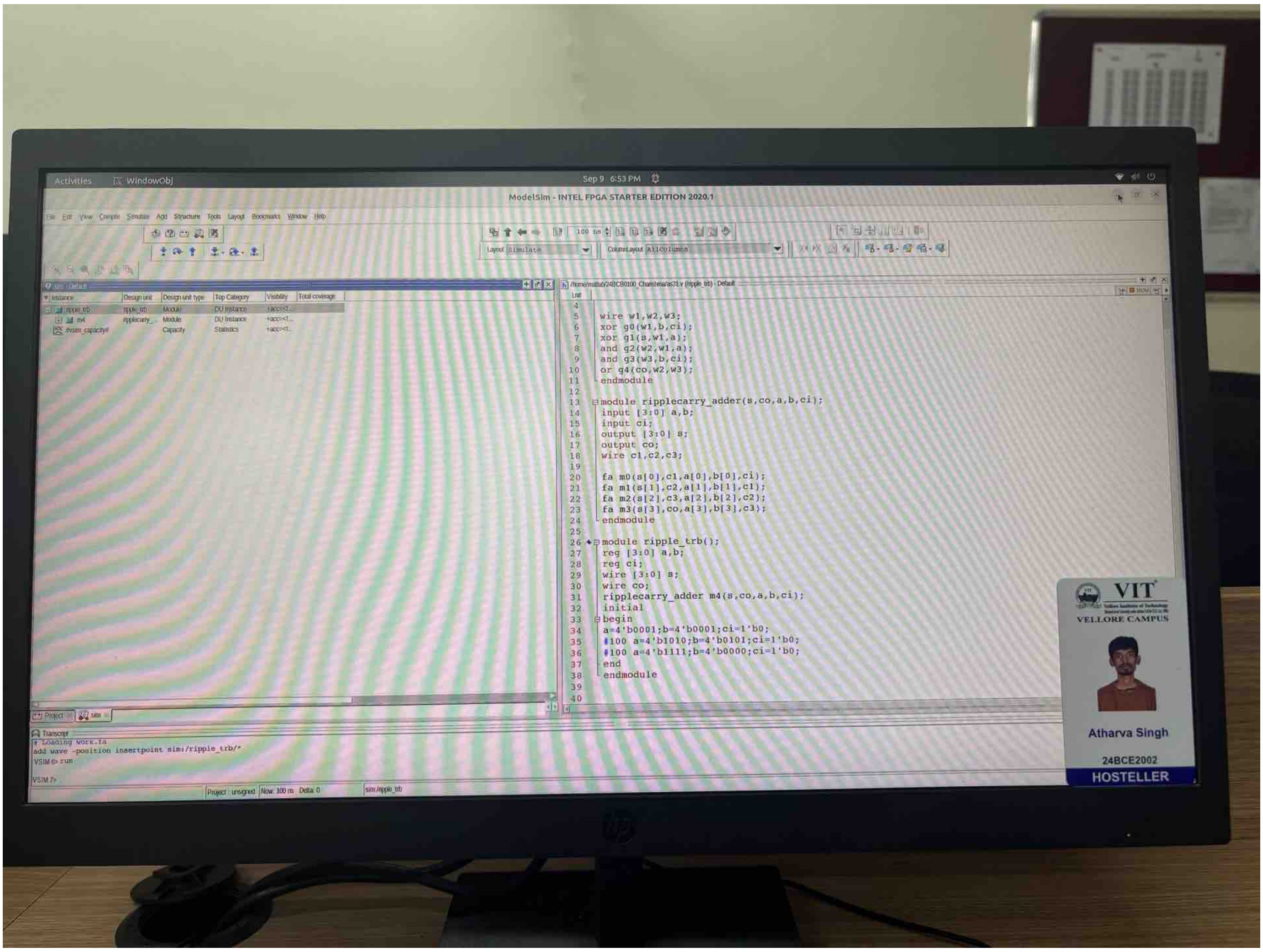
★ 4 bit ripple carry adder

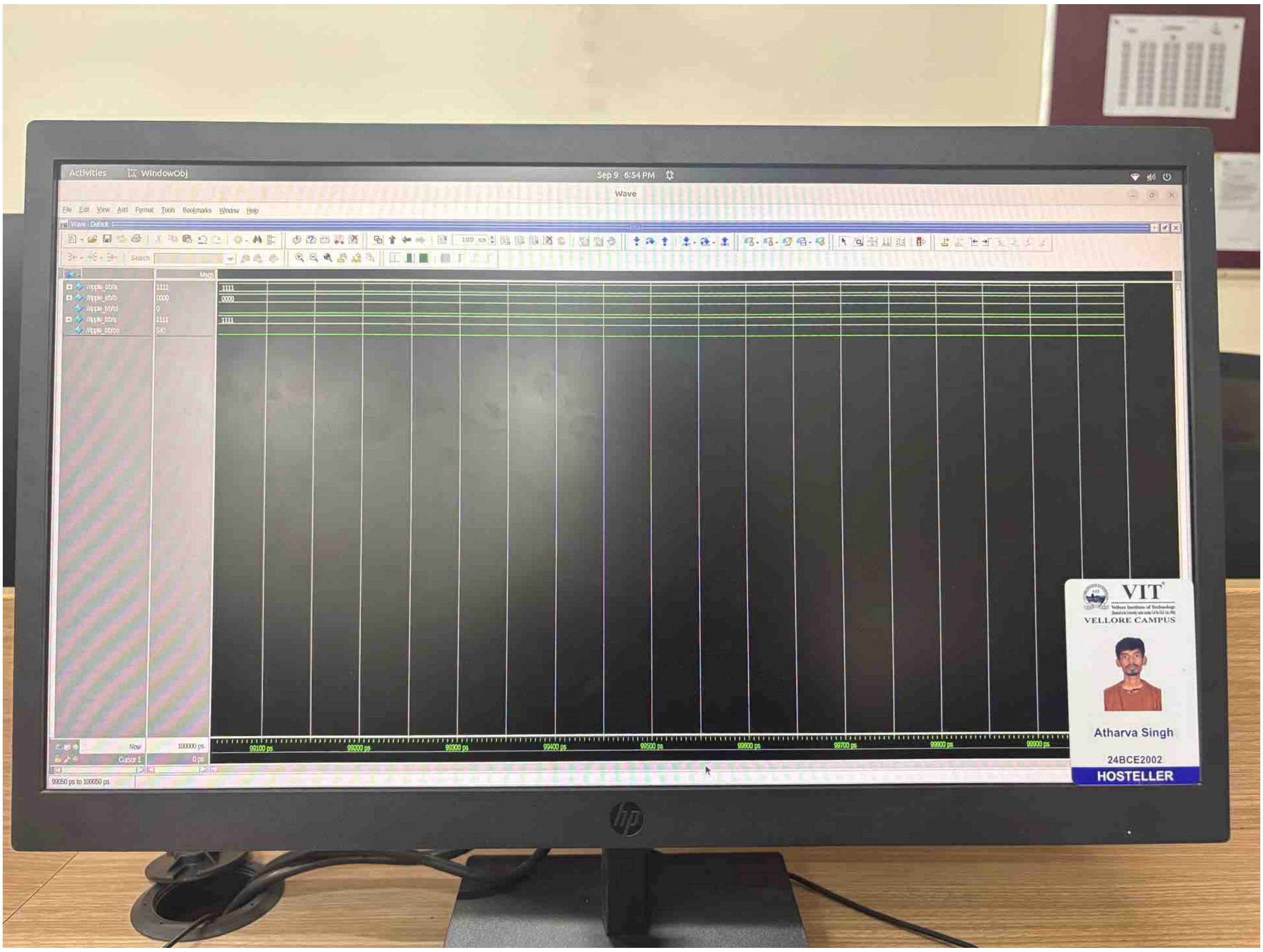
```
module full-adder(S,C,A,B,C in);
input A,B,C in;
output S,C;
wire w1,w2,w3;
XOR g0(w1,B,cin);
XOR g1(S,w1,A);
and g2(w2,w1,A);
and g3(w3,B,cin);
or g4(C,w2,w3);
endmodule
```




```
module ripplecarry-adder(s,c,a,b,cin);
input [3:0] a,b;
input cin;
output [3:0]s;
output c;
wire c1,c2,c3;
fulladder m0(s[0],c1,a[0],b[0],cin);
full-adder m1(s[1],c2,a[1],b[1],c1);
full-adder m2(s[2],c3,a[2,b[2],c2);
full-adder m3(s[3],c,a[3],b[3],c3);
endmodule

module ripple-tb();
reg [3:0]a,b;
reg cin;
wire [3:0]s;
wire c;
ripplecarry-adder m4(s,c,a,b,cin);
initial
begin
a = 4'b1100; b = 4'b1111; cin = 1'b0;
#100 a = 4'b1010; b = 4'b0101; cin 1'b0;
end
endmodule
```

O/P verified
9/9/25
24BCE2002


ModelSim - INTEL FPGA STARTER EDITION 2020.1
wire w1,w2,w3;
xor g0(w1,b,ci);
xor g1(s,w1,a);
and g2(w2,w1,a);
and g3(w3,b,ci);
or g4(co,w2,w3);
endmodule
module ripplecarry_adder(s,co,a,b,ci);
input [3:0] a,b;
input ci;
output [3:0] s;
output co;
wire c1,c2,c3;
fa m0(s[0],c1,a[0],b[0],ci);
fa m1(s[1],c2,a[1],b[1],c1);
fa m2(s[2],c3,a[2],b[2],c2);
fa m3(s[3],co,a[3],b[3],c3);
endmodule
module ripple_trb();
reg [3:0] a,b;
reg ci;
wire [3:0] s;
wire co;
ripplecarry_adder m4(s,co,a,b,ci);
initial
begin
a=4'b0001;b=4'b0001;ci=1'b0;
#100 a=4'b1010;b=4'b0101;ci=1'b0;
#100 a=4'b1111;b=4'b0000;ci=1'b0;
end
endmodule
VIT
VELLORE CAMPUS
Atharva Singh
24BCE2002
HOSTELLER



Activities
WindowObj
Sep 9 6:54 PM
Wave
VIT
VELLORE CAMPUS
Atharva Singh
24BCE2002
HOSTELLER

★ 3 bit magnitude comparator circuit

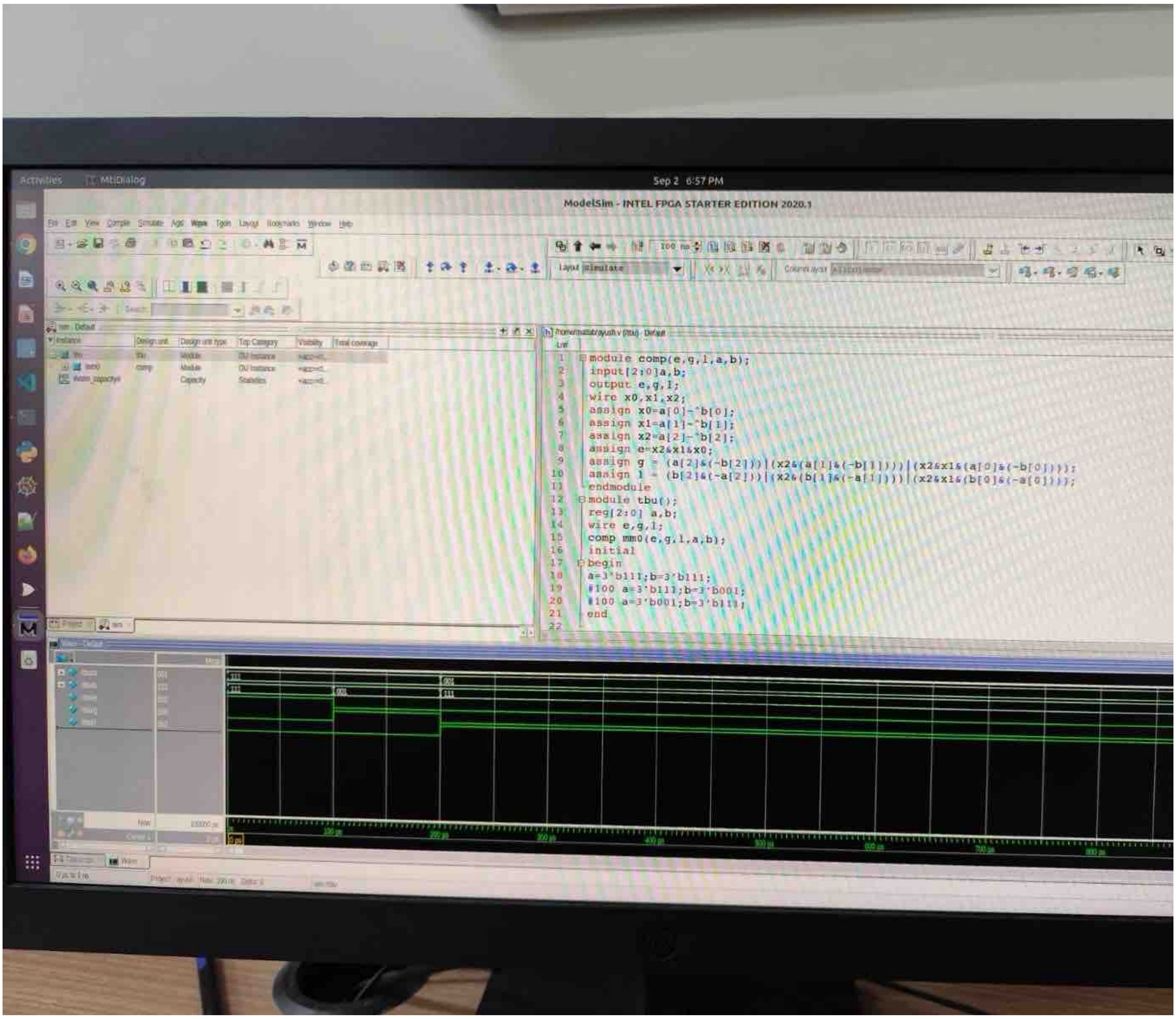
```
module com_3 bit (E,G,L,A,B);
input [2:0] A,B;
output E,G,L;
wire X0, X1, X2;
assign X0 = A [0]~^ B[0]
assign X1 = A [1]~^ B[1]
assign X 2 = A [2] ~^ B [2];
assign E = X 2 & X1 & X0;
assign G = (A [2] & (~B [2] )) | (x2 & ( A [1] & (~B[1]))) | (X2 & X1 & (A[0] & (~B[0])));
assign L = (B[2] & (~A[2])) | (X2 & (B[1] & (~A[1]))) | (x2 & x1 & B [0] & (~A[0]));
Endmodule

module tb_Comp();
reg [2:0] A,B;
wire E,G,L;
Comp_3 bit mm0 (E,G,L,A B);
initial
begin
A = 3'b 111 ; B = 3'b 111 ;
#100 A = 3'b 111 ; B = 3'b 001;
#100 A = 3'b001 ; B = 3'b111 ;
end
endmodule
```

O/P verified
9/9/25
24BEE2002

$A = A_2\ A_1\ A_0$

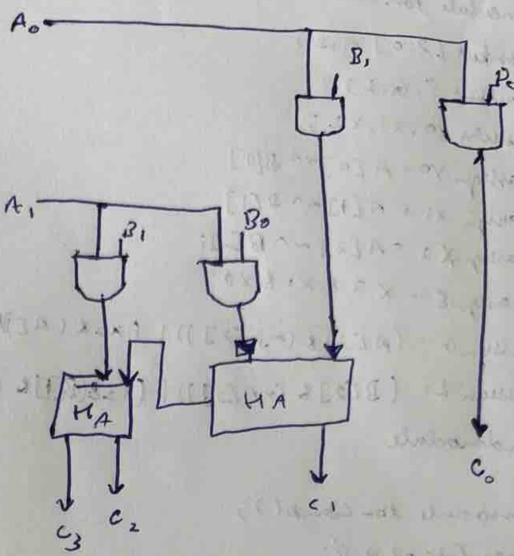
$B = B_2\ B_1\ B_0$

$A_2 A_1 A_0$	$B_2 B_1 B_0$	A > B	A = B	A < B
0 0 0	0 0 0	0	1	0
0 0 0	0 0 1	0	0	1
0 0 0	0 1 0	0	0	1
0 0 0	0 1 1	0	0	1
0 0 0	1 0 0	0	0	1
- - -	- - -	- -	- -	- -
1 1 1	1 1 0	1	0	0
1 1 1	1 1 1	0	1	0


Activities
MtiDialog
Sep 2 6:57 PM
ModelSim - INTEL FPGA STARTER EDITION 2020.1
module comp(e,g,l,a,b);
input[2:0]a,b;
output e,g,l;
wire x0,x1,x2;
assign x0=a[0]~^b[0];
assign x1=a[1]~^b[1];
assign x2=a[2]~^b[2];
assign e=x2&x1&x0;
assign g = (a[2]&(~b[2]))|(x2&(a[1]&(~b[1])))|(x2&x1&(a[0]&(~b[0])));
assign l = (b[2]&(~a[2]))|(x2&(b[1]&(~a[1])))|(x2&x1&(b[0]&(~a[0])));
endmodule
module tbu();
reg[2:0] a,b;
wire e,g,l;
comp mm0(e,g,l,a,b);
initial
begin
a=3'b111;b=3'b111;
#100 a=3'b111;b=3'b001;
#100 a=3'b001;b=3'b111;
end

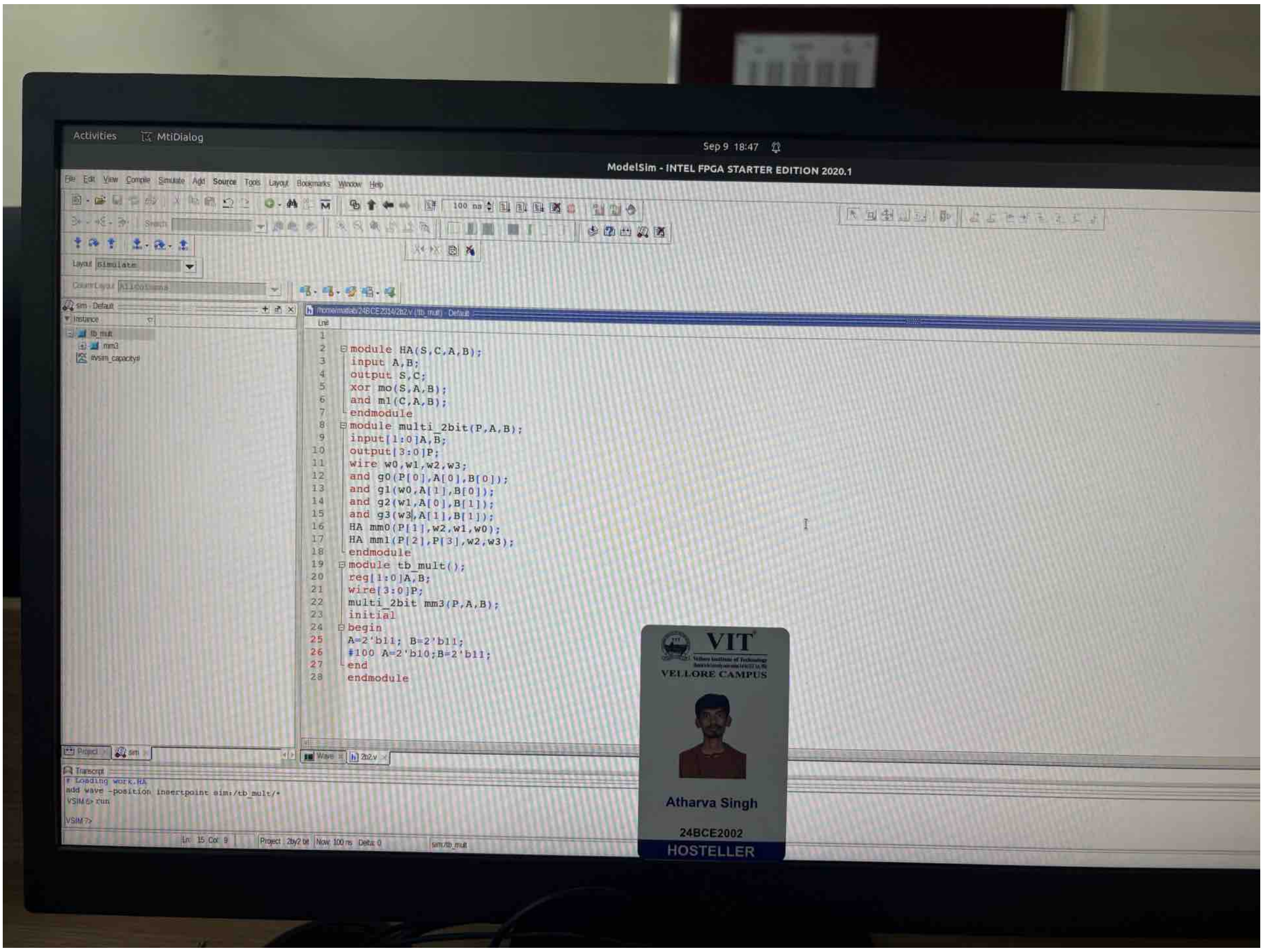
★ Two bit by two bit multiplier

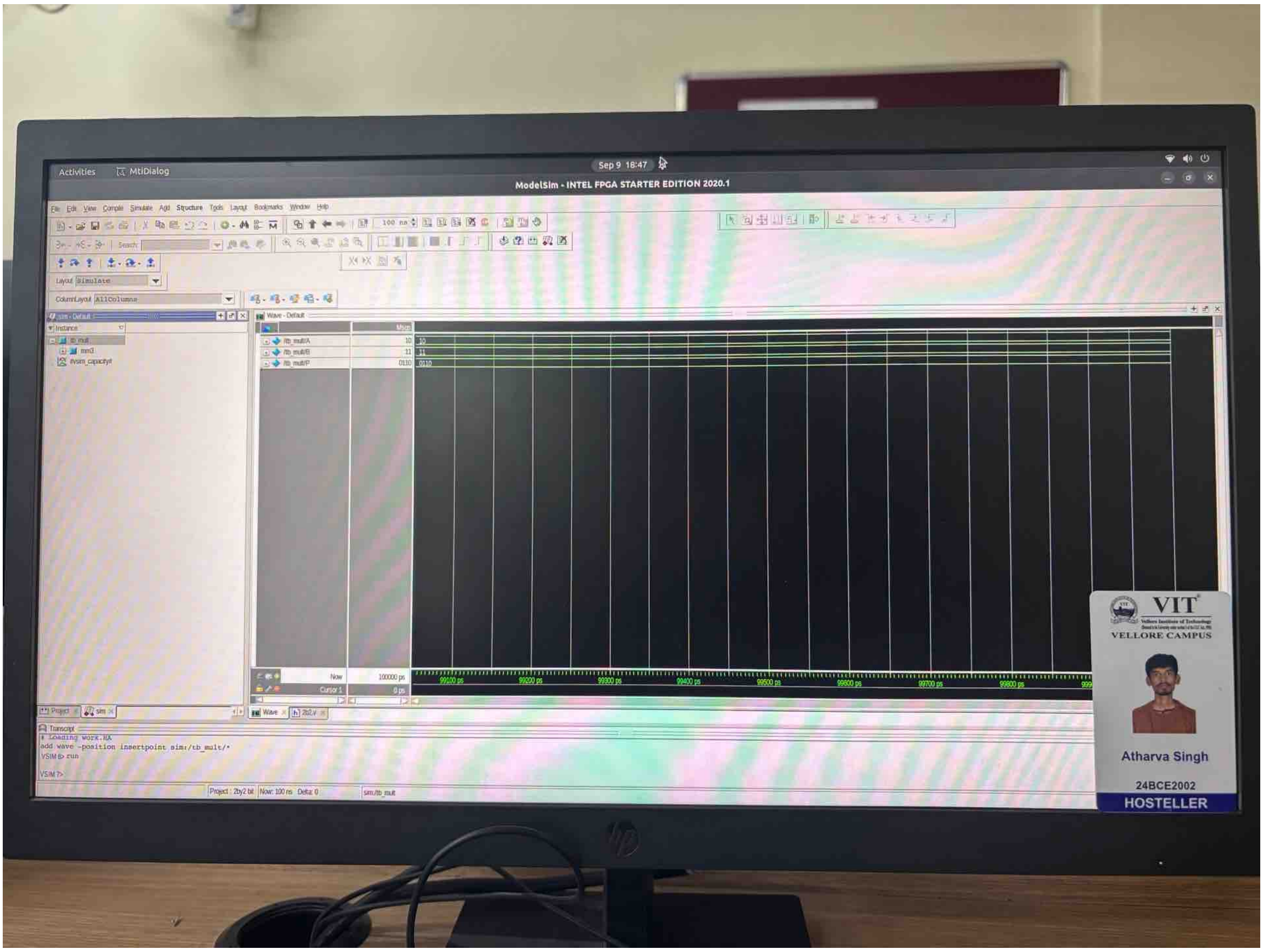
```
module HA(S,C,A B);
input A,B;
output S,C;
xor m0 (S,A,B);
and m1(C,A, B)
endmodule.
module tb();
reg A,B;
wire S, C;
HA m0(S,C,A B);
initial
begin
A = 1'b0; B=1'b0;
# 100 A = 1'b0; B=1'b1;
# 100 A = 1'b1; B= 1'b0;
# 100 A= 1'b1; B=1'b1;
end
endmodule
```




```
include "HA.v" module
mult_2bit (P,A,B);
input [1:0] A, B;
output [3:0] P;
wire w0, w1, w2, w3;
and g0(P[0], A[0], B[0]);
and g1 (w0, A[1], B[0]);
and g2 (w1, A[0], B[1]);
and g3(W3, A[1], B[1]);
HA m0 (P[1], w2, w1, w0);
HA m1 [P[2], P[3], w2, w3];
endmodule

module tb_mult();
reg [1:0] A, B;
wire [3:0] P;
mult_2bit m3 (P,A,B);
initial
begin
A = 2'b11 ; B = 2'b11;
# 100 A= 2'b10 ; B=2'b11;
end
endmodule
```

O/P verified
9/9/25
24BCE2002


Activities
MtiDialog
Sep 9 18:47
ModelSim - INTEL FPGA STARTER EDITION 2020.1
module HA(S,C,A,B);
input A,B;
output S,C;
xor mo(S,A,B);
and m1(C,A,B);
endmodule
module multi_2bit(P,A,B);
input[1:0]A,B;
output[3:0]P;
wire w0,w1,w2,w3;
and g0(P[0],A[0],B[0]);
and g1(w0,A[1],B[0]);
and g2(w1,A[0],B[1]);
and g3(w3,A[1],B[1]);
HA mm0(P[1],w2,w1,w0);
HA mm1(P[2],P[3],w2,w3);
endmodule
module tb_mult();
reg[1:0]A,B;
wire[3:0]P;
multi_2bit mm3(P,A,B);
initial
begin
A=2'b11; B=2'b11;
#100 A=2'b10;B=2'b11;
end
endmodule
add wave -position insertpoint sim:/tb_mult/*
VSIM 6> run
VSIM 7>
VIT
VELLORE CAMPUS
Atharva Singh
24BCE2002
HOSTELLER


Activities
MtiDialog
Sep 9 18:47
ModelSim - INTEL FPGA STARTER EDITION 2020.1
VIT
VELLORE CAMPUS
Atharva Singh
24BCE2002
HOSTELLER